

OpenHarness + NVIDIA OpenShell

Remote-agent confinement | Integration design

Decision: Keep planning and orchestration in OpenHarness. Deploy an OpenShell gateway beside the remote compute, and let its compute driver provision an isolated remote-agent workload and its supervisor. Delegate work over MCP; use the TypeScript SDK only for the infrastructure lifecycle.

Prepared 28 September 2026 for [implementation issue #67](#). Status: proposed design; no runtime implementation or deployment is included. OpenHarness source reviewed at [1919c4f](#); OpenShell baseline is release [v0.1.2](#), commit [6648bd0](#). [S1, R1]

Where the pieces belong

Component	Responsibility in this design
OpenHarness runner	Plans, retrieves knowledge and calls its existing MCP tools. The remote agent is another tool destination.
OpenHarness API	Owns remote-target configuration, authorized delegation, sandbox lifecycle, cancellation and trace correlation.
OpenShell gateway	Remote control plane: persists sandbox state and policy, authenticates clients, coordinates compute and provider credentials.
OpenShell supervisor	Trusted process outside the workload. Evaluates remote-agent egress, applies credential injection and relays access.
openshell-sandbox	Runtime inside the workload. Owns the agent process and applies the local confinement mechanisms.
Remote agent	An autonomous process plus an MCP server in the workload. Runs its own reasoning loop within the admitted role policy.

The last three roles reflect the pinned OpenShell architecture. The supervisor is not a daemon to add to the OpenHarness Compose stack. Its placement and channel are managed by the OpenShell compute driver. [S2, S3]

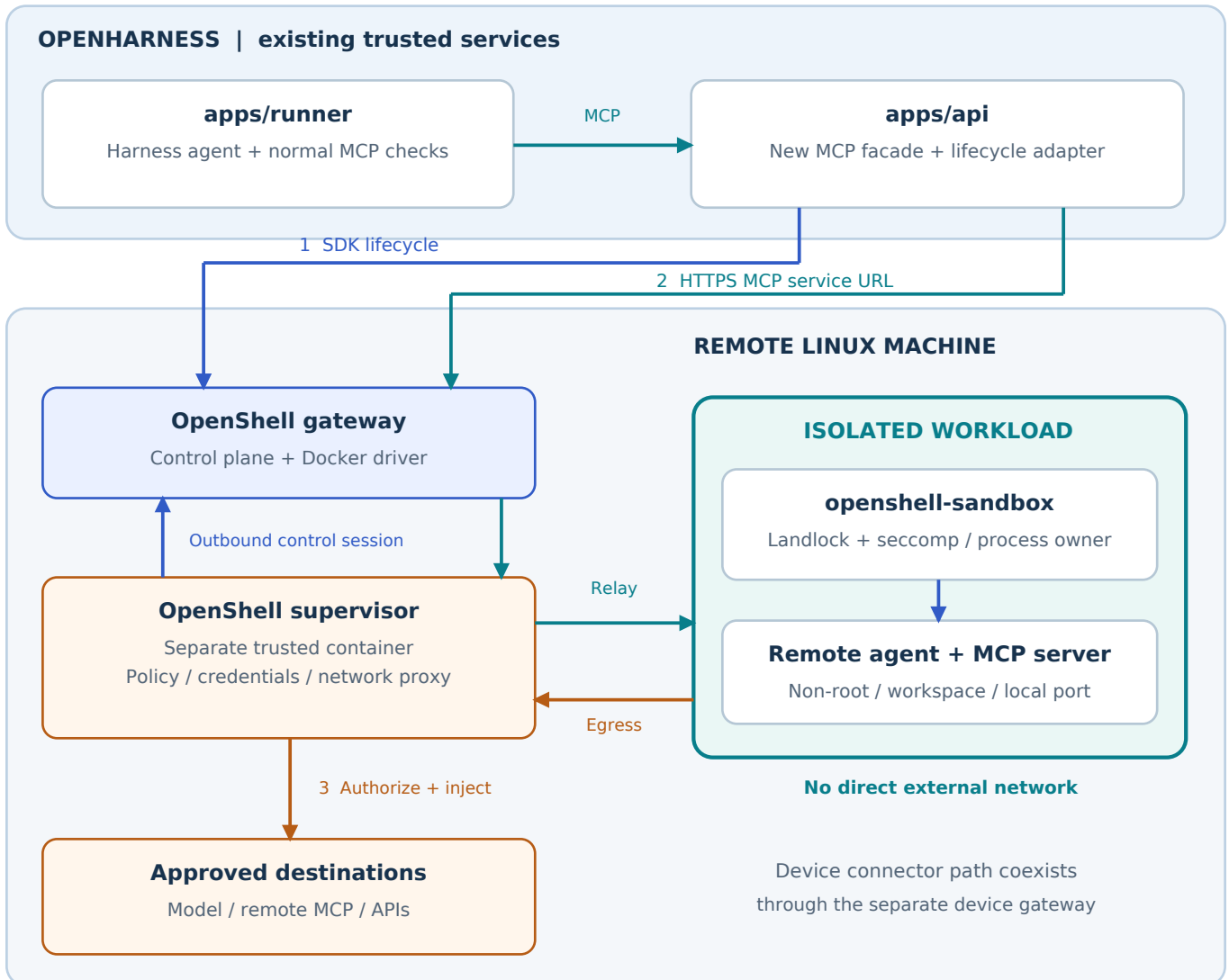
Three paths to keep distinct

Control: OpenHarness API → OpenShell gateway via the SDK. **Delegation:** harness MCP call → API delegation endpoint → remote agent's exposed MCP service. **Remote egress:** agent → sandbox runtime → supervisor → approved model, MCP servers or APIs.

The existing **device gateway** in [gateway/](#) remains the WebSocket/MCP hub for typed connector operations. A machine may host both a connector and an OpenShell-managed workload. Privileged GPU diagnostics stay on the connector path. [R1, R2]

Deployment and trust boundaries

This is the recommended first deployment: a remote Linux machine with OpenShell's Docker compute driver. A Kubernetes deployment has the same responsibilities, but different runtime placement and networking. The drawing describes the Docker baseline. [S2]

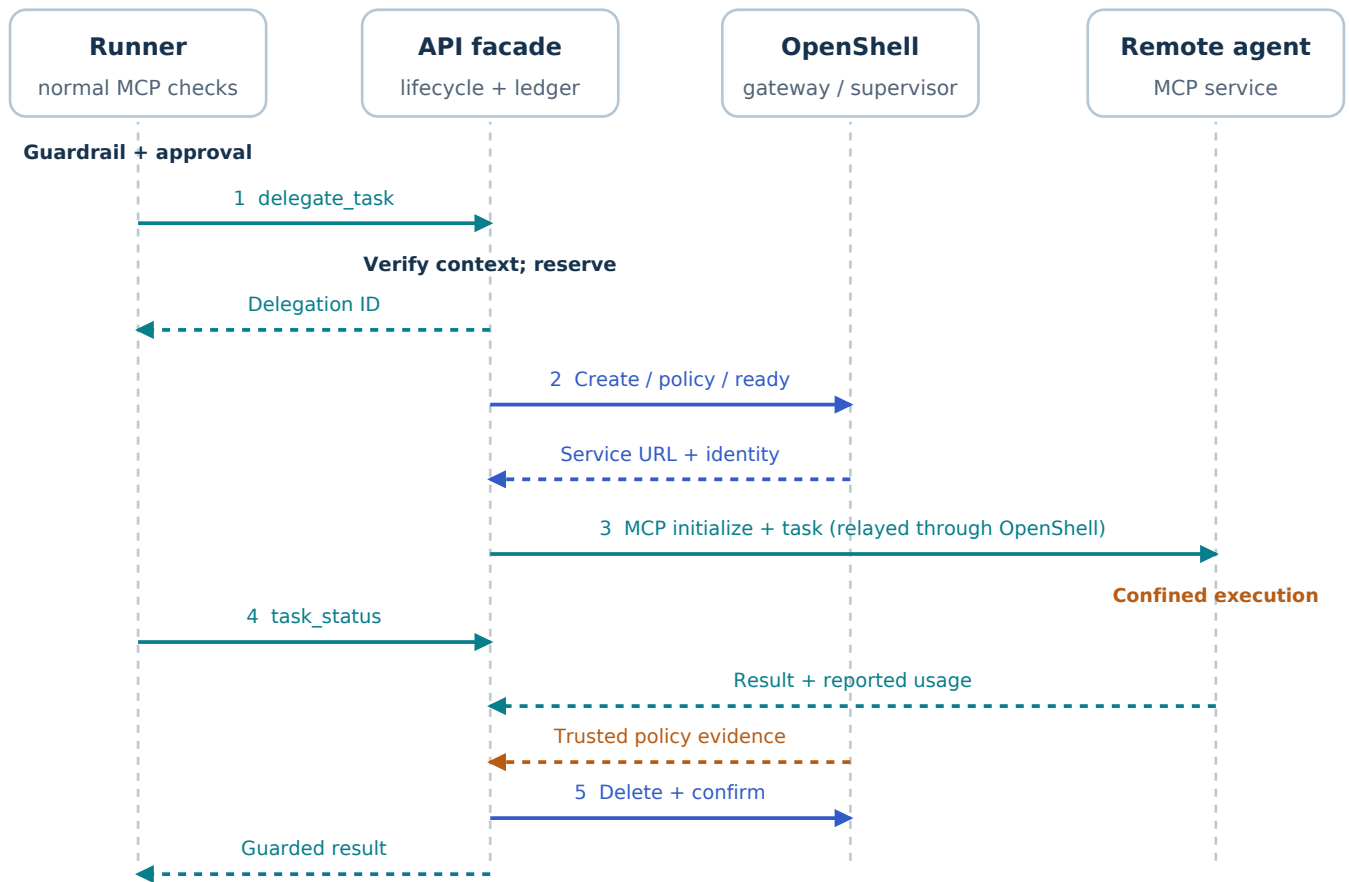


Blue lines carry lifecycle and configuration. **Teal lines** carry MCP task traffic and service relays. **Orange lines** carry policy-controlled remote egress. The supervisor's outbound session lets the OpenShell gateway relay into the workload without a public workload port. [S3]

The workload has no direct external network attachment in this baseline. The driver provisions a protected channel to the supervisor. Provider credentials and management credentials belong outside the agent workload. Workload-local loopback remains available for its MCP server. [S2, S4]

Reachability requirement: OpenHARNESS must reach the remote OpenShell gateway and its service-routing domain over authenticated TLS, through an operator-managed private route or ingress. Installing OpenShell alone does not reuse the device connector's dial-out tunnel. Do not put a Docker socket, host root filesystem or device-gateway admin token into the workload.

One delegated task, end to end



Recommended MCP surface

Add a stable, tenant-scoped MCP endpoint in `apps/api`, registered through the existing connection mechanism. Its tool catalog is known before a run: `delegate_task`, `task_status` and `cancel_task` are proposed names. The API provisions the task's sandbox and forwards the task as MCP to the workload service. This adds an MCP facade, not another external agent-tool protocol.

Use `delegate_task` to return a durable delegation ID promptly. Poll bounded status/result calls for long jobs; do not hold one tool call open for the whole autonomous run. Discovery must have no provisioning side effects. A managed persistent sandbox can also be an ordinary direct MCP connection, but the facade is the selected default for per-task lifecycle and budget accounting.

The remote image must implement this MCP contract; OpenShell does not supply the agent. Result fields should include status, summary, bounded artifact references and reported usage. Never accept management commands, policy edits, gateway URLs or executable images from a task argument.

The green return path passes through the existing tool-output guardrail. Streamed progress and downloaded artifacts must be filtered or withheld before entering model context, too. Failure must produce a structured tool error, not a fabricated successful answer.

The integration contract

API owns infrastructure; MCP carries work

Put a release-specific `OpenShellAdapter` behind an interface in `apps/api`. Suggested operations are `checkTarget`, `createTaskSandbox`, `waitReady`, `getServiceUrl`, `observe`, `stop` and `deleteAndConfirm`. These are our proposed interface names, not NVIDIA SDK method names.

Verified SDK building blocks include `OpenShellClient.connect`, `health`, `sandbox.create`, `sandbox.waitReady`, `sandbox.delete` and `sandbox.waitDeleted`. Create supports `serviceExposures` and returns `serviceUrls`; raw clients cover additional gateway RPCs. The SDK requires Node 20.3+, which fits this repository's Node 22+ baseline. It is distributed through GitHub Packages. [S5, S6]

Supply the explicit OpenShell workspace on every operation. Admit the complete static filesystem/process policy at creation. Treat the returned service URL as untrusted configuration: validate its scheme, expected service domain, resolved address and TLS identity before attaching credentials. Use the MCP HTTP transport already in `packages/core/src/mcp.ts`; explicitly allow only required private hosts through existing URL safeguards. [R3]

Separate the identities

Boundary	Proposed identity and authorization
Runner ↔ API MCP facade	Internal, short-lived delegation capability bound to tenant, run, tool call, target, approved input digest and deadline. Mint after normal guardrail/approval checks.
API ↔ OpenShell gateway	Operator-provisioned service identity scoped to the selected workspace and required lifecycle operations. Store encrypted credential references.
API ↔ exposed MCP service	Gateway-supported service authentication, with distinct least-privilege access where available. Verify credential stripping before workload delivery.
Supervisor ↔ model/tool endpoint	OpenShell-managed provider credential injection for explicitly approved destinations. No raw provider secrets in task input or artifacts.

A static MCP connection token alone proves neither run ownership nor approval. Add trusted run context in transport metadata and verify it against durable records; never trust model-supplied `runId`, budget or owner fields. Existing dispatch checks remain the authorization gate. Only the new managed connection type needs this context hook. [R3, R4]

Use noninteractive authentication. An existing operator-managed OIDC service account is one upstream option; Kubernetes requires OIDC or a trusted access proxy for user identity. Do not add federated Studio login. Public health success is not proof that lifecycle credentials work: exercise an authorized capability as well. [S7]

Before coding: prove service auth, token refresh, POST/GET/DELETE MCP transport and cancellation against the pinned deployment. If service auth needs an HTTP credential provider or mTLS support absent from `connectMcp`, add that scoped transport support; do not fall back to unauthenticated access or SDK exec as a tool path.

Policy and budget boundaries

Author one policy per remote-agent role

Choose policy from an operator-managed role catalog and snapshot its revision/hash for every delegation. Do not synthesize it from OpenHarness connections. The remote agent's own destinations, filesystem and runtime identity define its policy. Revalidate the effective admitted policy, including OpenShell gateway overrides, before sending a task.

Control	Design requirement
Filesystem	Workspace and necessary scratch paths writable; required toolchain paths readable. No host home, secrets, container socket or broad host mounts. Validate required paths actually exist.
Kernel and identity	Require Landlock ABI v3+. Set landlock.compatibility: hard_requirement . Use a non-root workload identity and verify the driver enforces it.
Network	Allow the role's model endpoint, necessary APIs and remote MCP destinations only; restrict inspected requests and calling binaries. Pin each remote MCP endpoint's mcp.versions .
Credentials	Attach only role-required providers. Keep actual endpoint credentials at the trusted injection layer. Bound credential lifetime to delegation where supported.
Resource limits	Set driver-supported CPU, memory and storage limits and an externally enforced wall-clock deadline. Cap request size, result size and concurrency.

In this release, [process.run_as_user](#) and [run_as_group](#) apply to Docker/Podman; Kubernetes and VM identity comes from driver configuration. Filesystem/process policy is create-time state. Audit/enforce is an endpoint request-rule setting; audit allows rule violations. Perform audit discovery only on controlled fixtures, then require enforce for accepted workloads. [S8, S9]

What existing OpenHarness checks do—and do not—cover

The normal tool-input checks, human approval and tool-output checks cover the delegation boundary. They do not approve every command the autonomous remote agent will choose. OpenShell MCP inspection matches methods/tool names, not arguments or returned content; an allowed remote tool still needs suitable server-side permissions. [S9, R4]

The parent's tool-call budget is not a hard token cap on a separate agent's model calls. Reserve a child allowance in a durable ledger before dispatch; bound it by the parent's remaining allowance and the target maximum. Require the controlled remote-agent implementation to stop at its allocation and report usage. Reconcile usage before releasing the reservation; retain reservations on unknown outcomes.

For a hard cap against a compromised workload, enforce model usage outside it using provider quotas or an approved metering proxy. Self-reported usage is insufficient. The first implementation must label whether its token limit is cooperative or externally enforced. Wall-clock expiry must stop/delete the workload even if its MCP server stops responding.

Lifecycle, recovery and evidence

A durable record outlives the HTTP request

Proposed delegation states: `reserved` → `provisioning` → `ready` → `running` → `collecting` → `deleting` → `complete`. Failures enter `failed` with a separate cleanup state of `pending`, `confirmed` or `failed`. Distinguish task success from resource cleanup success in the UI and trace.

Persist the reservation and a unique sandbox name before create. Record gateway, workspace, sandbox ID/generation and operation key as soon as known. A lost create response requires lookup/reconciliation, not an immediate second create. Use unique names and verify immutable identity before destructive operations so retries cannot delete a replacement sandbox.

Event	Required behavior
Approval denied / input blocked	Return through the existing tool machinery. Do not create a sandbox or spend the child reservation.
Provisioning / MCP readiness failure	Record the cause, delete any created workload, confirm cleanup and release only unused reservations. Sandbox Ready alone is not MCP readiness.
Runner/API crash or ambiguous delivery	Reconcile the same delegation ID and remote operation key. Query existing task status; do not replay non-idempotent work blindly.
Cancellation / deadline / budget exhaustion	Send MCP cancellation when possible, then enforce stop/delete through the API adapter. Record cancellation separately from verified termination.
OpenShell gateway unavailable	Stop admitting new delegations. Mark cleanup pending and retry from durable state. Do not claim existing workloads stopped merely because control-plane access failed.
Persistent target	Lease exclusively per owner/run by default; reset task state between uses. Retain sandbox intentionally, release lease and detach task-scoped credentials.

Exposed services cannot be created with `--expose` plus `--no-keep`. For task sandboxes running an MCP server, explicitly delete after collecting the bounded result and evidence. Handle asynchronous deletion and confirm the original sandbox ID is gone. A periodic reconciler handles abandoned leases and retries cleanup after restarts. [S4, S6]

Make confinement visible in the run trace

Ingest OpenShell lifecycle and policy events through the API adapter. Proposed trace events: `remote_agent_started`, `remote_policy_denied`, `remote_agent_completed` and `remote_cleanup_pending`. Include delegation ID, sandbox ID, policy revision, enforcement mode and timestamps; redact credentials and sensitive request material.

Record supervisor policy evidence separately from agent claims. A tool saying “permission denied” is not proof of enforcement. Preserve relevant events before deleting the workload. If a log gap prevents proving denial, show an evidence gap and fail the acceptance assertion. Reuse the existing run/event persistence seam rather than inventing a second user-visible run history. [R4]

Changes within this repository

All entries below are proposed implementation seams. The current repository has machine/cluster routes and MCP connection records, but no remote-agent target implementation. Machines and clusters are administered through the device gateway, so copy their UI pattern, not their storage ownership. [R2, R3]

Location	Proposed change
apps/api/src/remoteAgents.ts (new)	Tenant-scoped target CRUD, check/connect, managed MCP facade and lifecycle routes. Register through the existing API auth boundary.
apps/api/src/openshell/ (new)	Pinned SDK adapter, credentials, readiness checks, log ingestion and durable cleanup reconciler.
packages/core/src/schema.ts	Remote target and delegation schemas; managed connection ownership metadata. Preserve existing MCP bindings and tool allow-lists.
New repository interfaces + db.ts	RemoteAgentTargetRepository and RemoteDelegationRepository , with Mongo implementations and in-memory test doubles. Index owner, operation key, active lease and cleanup state.
runtime.ts , mcp.ts , runs.ts	Trusted delegation context, parent reservation/accounting and cancellation wiring through normal MCP dispatch and events. Keep SDK lifecycle out of the runner.
apps/studio/src/components/	Remote Agents page beside Machines/Clusters, target detail and run trace links.
deploy/openshell/ (new), docs/openshell.md	Versioned remote deployment assets, policy catalog, agent image recipe and operator guide. No OpenShell service in production OpenHarness Compose.

Minimum persisted model

RemoteAgentTarget: ID, owner/tenant, display name, optional machine/cluster association, gateway URL, encrypted credential reference, explicit OpenShell workspace, image digest, role-policy reference and hash, provider references, task/persistent mode, default budgets and concurrency, enabled flag, validated release/capabilities and last health result.

RemoteDelegation: owner, target, parent run/tool call, idempotency key, approved-input digest, budget reservation, deadline, sandbox name/ID/generation, effective policy hash, service endpoint, remote task ID, lifecycle state, actual/reported usage, log cursor, cleanup state and terminal result reference. Never persist task credentials in this record.

Studio should display two separate statuses: “Connected” for the control plane, and “Ready to delegate” after policy, identity and MCP readiness checks. Show audit mode prominently and distinguish typed machine operations from autonomous remote-agent work. Present target budgets and policy revision before use; show cleanup failures after the task ends.

The release's SDK package metadata in source uses a build-time placeholder version. Resolve the actual published package for [v0.1.2](#), pin it and its integrity in the lockfile, and match gateway/runtime artifacts by digest. Keep registry configuration and build instructions in-repo; provide package credentials through secrets, never a committed token. [S5, S6]

Release findings and rollout

Corrections to the original issue sketch

Issue assumption	Verified finding for the design baseline
Supervisor lives inside the agent container	Trusted supervisor and workload runtime are separated. Draw and deploy them as separate trust zones. [S2]
Ephemeral MCP service can use <code>--no-keep</code>	Service exposure keeps a listening workload; the CLI rejects this combination. Use explicit cleanup for the chosen MCP lifecycle. [S4]
Loopback is always blocked	Proxy egress to loopback/link-local/metadata addresses is blocked; sandbox-local loopback service access is allowed. A host model service needs a policy-approved routable address. [S9]
Dependency makes no default outbound calls	Telemetry is default-on. Set <code>OPENSHELL_TELEMETRY_ENABLED=false</code> on the OpenShell gateway or use a verified telemetry-free build. Capture traffic to verify the deployed configuration. [S10]
All remote budgets apply automatically	Existing checks govern the parent tool boundary. Remote model usage needs reservation, accounting and an explicit enforcement mechanism. [R4]

Implementation order for #67

- 1. Prove the release contract.** Verify Landlock, package availability, service forwarding and telemetry opt-out on isolated test compute. Pin artifacts and build the remote MCP agent image in-repo.
- 2. Deliver the vertical slice.** Add target storage, adapter and MCP facade. Run a task under enforced policy, ingest evidence and confirm deletion.
- 3. Complete orchestration.** Add trusted context, accounting, cancellation, idempotency, cleanup recovery, Studio controls and [docs/openshell.md](#).
- 4. Verify implementation.** Run typecheck, unit tests and isolated real-stack integration. Keep hosted CI disabled. This design provides no runtime acceptance evidence.

Acceptance must prove the boundary

- Exercise the remote process directly: allow one action and deny another through OpenShell, with application checks permissive. Correlate trusted denial evidence with the run trace. Verify a forbidden filesystem write leaves no file.
- A denied approval/guardrail creates no workload. Filter malicious output. Reject invalid tenant/run capabilities and concurrent budget over-reservation.
- Inject restart, failed creation, response loss, cancellation and cleanup retry only in the isolated test Compose project. Assert no duplicate task, leaked workload or deletion of a replacement sandbox. Its OpenShell test compute must also be exclusively owned.
- Verify no real secrets or direct network escape in the workload. Capture idle/start/stop traffic with telemetry disabled; distinguish configured traffic from unsolicited outbound calls.

Remaining deployment inputs: actual target endpoint, operator authentication mode, selected remote-agent implementation and model quota support. These determine configuration and hard-budget guarantees; they do not change the component split above.

Evidence and source links

Research date: 28 September 2026. NVIDIA links below are pinned to [v0.1.2](#) except the release record. Recheck deployed capabilities before implementation; do not mix 0.0.x and 0.1.x assumptions. Source inspection verifies API/design facts, not runtime behavior on the intended remote machine.

NVIDIA primary sources

- **S1 — Release:** [OpenShell v0.1.2](#), commit [6648bd0c290efbc41ba131ee9831ee45cd431f94](#).
- **S2 — Runtime boundary:** [architecture/sandbox.md](#). Supervisor/runtime split, process confinement, network isolation and authenticated private channel.
- **S3 — Control plane:** [architecture/gateway.md](#). State, compute drivers, credentials and supervisor relays.
- **S4 — Sandbox services:** [Sandbox overview](#). Service exposure, returned service URLs and ephemeral lifecycle caveats.
- **S5 — SDK distribution:** [TypeScript SDK documentation](#) and [package manifest](#). GitHub Packages and runtime requirements.
- **S6 — SDK contract:** [SDK README](#). Create-time policy, raw client, workspace selection, renewable credentials and deletion results.
- **S7 — Authentication:** [Gateway authentication](#). Local mTLS, service automation, deployment constraints and health versus auth.
- **S8 — Policy fields:** [Policy schema](#). Landlock compatibility, identity, endpoint enforcement and MCP revisions.
- **S9 — Network semantics:** [Network rules](#). Audit behavior, destination restrictions, loopback exception and MCP inspection limits.
- **S10 — Outbound telemetry:** [Telemetry](#). Default-on collection, runtime opt-out and build-time exclusion.
- **S11 — Migration context:** [Upgrade to 0.1.0](#). Breaking policy, deployment and API changes; coordinated upgrade required.

OpenHarness evidence

- **R1 — Scope:** [Issue #67](#). Remote agents only, MCP-only tools, remote control plane, acceptance and non-goals.
- **R2 — Existing remote path:** [Device architecture](#), [apps/api/src/devices.ts](#), [apps/api/src/clusters.ts](#) and [gateway/](#). Routes and source were checked alongside the architecture document.
- **R3 — MCP and storage:** [MCP client](#), [schema.ts](#), [security.ts](#) and [db.ts](#). Existing connection shape, encryption, URL checks and collection boundaries.
- **R4 — Execution:** [Runtime](#), [runs.ts](#), [human.ts](#), [guardrails.ts](#) and [docs/subagents.md](#). Approval/guardrail dispatch, parent budgets, cancellation and trace model.

The API facade, record schemas, capability binding and reconciliation workflow are OpenHarness design proposals derived from these sources. They are not claims that OpenShell or this repository already implements those features.